Paper Summary 1

Mohammad Kaosain Akbar

(UCID: 30136155)

Paper Title:

The Promise and Reality of Continuous Integration Caching : An Empirical Study of Travis CI Builds

Paper Reference (ACM Format):

Taher A. Ghaleb, Daniel Alencar da Costa, and Ying Zou. 2026. The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds. Presentation at the 30th International Conference on Evaluation and Assessment in Software Engineering (EASE '26), Glasgow, Scotland, United Kingdom.
https://ui.adsabs.harvard.edu/abs/2026arXiv260119146G/abstract

Paper Summary:

In this paper the authors discussed how Continuous Integration (CI) caching of Software Engineering is actually used in various software projects and whether caching artifacts of CI improves build time which developer expects. Typically, CI caching stores artifacts and information such as downloaded dependencies, build reports, etc., so that subsequent builds can reuse these artifacts instead of downloading or generating them again. Although its thought that caching accelerates CI processes, the findings of the paper highlights that the practical benefits, maintenance and potential over head of CI caching have not been studied to full extent. This research tries to seek answer for five main questions such as why projects adopt CI caching, how these caches are maintained, whether caching reduces build duration, what are the potential overhead of CI caching and what are the typical problems the developers encounter for using CI caching.

For this research the authors analyzed 513384 Travis CI builds from 1279 Github projects in Java and Ruby. Additionally, the authors also use repository mining, CI build logs, Travis CI issue reports, multiple statistical modeling and developer feedback. They also submitted 85 pull requests which enabled caching in projects that had not previously used CI caching.

The initial finding of the paper mentions out of 1279 projects only 380 of them (30%) used CI caching. Therefore, this means that caching adoption is significantly low. Among these 380 projects, those with longer build, more dependencies, higher number of developers and development activities were more likely to adopt CI caching. Authors concluded that lack of awareness was an important reason for non-adaptation or late adaptation. They also performed caching enabled pulling request in 85 projects where 41 of the project maintainers accepted the pull or merge request which indicates that some projects benefit from CI caching but developers did not configure it initially.

The second finding of the paper deals with cache maintenance. Among the 380 projects that used caching, 24% of them maintained cache configuration and remaining 74% cached once and never changed it. The authors mentioned that developers often find it difficult in knowing what should be cached and thus end up performing trial and error on caching of various CI artifacts. The authors also suggest that CI platforms should include tools that identify frequently and infrequently changing artifacts so that these artifacts can be cached or not cached, respectively.

The third finding suggests that caching does not always make CI faster. Among the 380 projects which performed CI caching, only 33% experienced significant reduction in build duration. From the remaining projects where caching was not used, 50% had no significant change and only 17% had an actual increase in the build time. Based on this finding, the authors concluded that caching cannot be considered as one single factor to optimization.

Another finding of this research paper discussed about studying the overhead due to CI caching. Cache upload occurs six times more than the cache download time. The frequent cache upload indicates that there are often unnecessary artifacts within those CI cache files. Moreover, among the 380 projects with CI caching, 27% of the projects contain cached dependencies which are no longer being used, pointing out that cache often store stale artifacts over time.

Final finding of the paper highlights that software developers commonly deal with corrupted and outdated caches, CI dependencies that were not correctly updated and conflicts between caches used by different branches. Authors suggested that clearing the caches resolved around 62% of the analyzed reported issue.

Overall, this research paper shows the importance of not just simply enabling caching but also deciding which artifacts should be cached and continuously maintained as the project evolves. Poor CI cache selection can create unnecessary uploads, store stale artifacts, maintenance effort and longer build times. Therefore, the author believes that effective CI caching requires context aware approach that identify useful caching files and avoid any artifacts which changes too frequently.

Paper Relevancy Towards My Research:

This paper is directly related to my CI/CD research especially in understanding CI caching and identifying what artifacts and dependencies in CI pipelines are suitable for caching. The authors of this paper shows that caching is not always a universal solution for optimizing CI/CD pipelines and often does not significantly reduce build time. These findings can contribute to my broader research where I will be focusing on using AI to optimize CI/CD pipelines, build systems, and other DevOps-related decisions.